

商保电子凭证智能服务平台

接口规范 V3.0

博思云易
2025 年 3 月

文档修订记录

版本号	修订日期	修订概述	修订人	备注
V3.0.0	2025-03-03	版本创建	赵昊	
V3.0.1	2025-04-21	4.6.4:增加流水号 4.5.2、4.5.3、4.5.4、4.5.5、4.5.6: 支持税票收单 4.5.7:支持多张票据图片	赵昊	
V3.0.2	2025-04-25	附录 7、附录 10:增加门诊病历、 住院病案首页、入院记录/入院 小结	赵昊	
V3.0.3	2025-05-29	增加 4.7.1 个人健康评测 调整 4.6.1、4.6.2 中既往风险评 测结果结构 附录 1:增加是否医保票据字段， 医疗类别增加慢特病门诊 4.6.4 案件状态同步:案件状态增 加 拒赔 4.5.2 票据五要素收单:支持税 票查验 新增：4.5.11：票据报销状态查 询 4.5.9 票据状态查询:支持税票	赵昊	

目录

1. 使用范围	4
2. 接口协议	4
2.1. 访问协议	4
2.2.HTTP 参数规范	5
2.1.1. 服务请求方发起请求	5
2.1.2. 服务提供者返回(响应)请求结果	7
2.3.签名规则	9
2.3.1.签名与加密算法	9
2.3.2.请求随机标识算法	11
2.4.数据签名与加密示例	12
2.4.1.BASE64-MD5 数据编码与签名示例 V2.0	12
2.4.2.BASE64-SM4-SM2 数据加密与签名示例 V3.0	14
2.5.结果标识列表	14
3. 接口列表	15
4. 接口详情	15
4.1. 个人授权	15
4.1.1. 个人授权备案	15
4.2. 个人健康评测	18

1. 使用范围

- ❖ 适用业务：商业保险相关业务
- ❖ 适用场景：商保核保、理赔
- ❖ 适用对象：商保公司

2. 接口协议

2.1. 访问协议

本接口规范基于 HTTP 协议的 REST 服务方式进行交互，提交方式全部使用 POST；

传输方式	支持 HTTP/HTTPS 传输
提交方式	HTTP “请求-响应” POST
数据传输格式	提交和返回数据为 JSON 格式（Content-Type: application/json）
字符编码	统一采用 UTF-8 字符编码
签名算法	根据 API 版本号选择： API 版本号 2.0 版本使用 MD5 签名 API 版本号 3.0 版本使用 SM2 签名(私钥签名,公钥验签)
数据加密	根据 API 版本号选择： 2.0 版本使用 BASE64 将传输数据进行编码 3.0 版本使用 SM4 国密加密算法将传输数据进行加密
签名要求	详细方法请参考 2.3 签名规则
API 版本号	2.0(MD5 签名、BASE64 编码 data 传输参数) 3.0(SM2 签名、SM4 加密 data 传输参数)

2.2.HTTP 参数规范

2.1.1. 服务请求方发起请求

- 请求 URL: [https://\\${ip}:\\${port}/ciras-rest/ins-cl/\[rest-api\]](https://${ip}:${port}/ciras-rest/ins-cl/[rest-api])
 - rest-api 参见具体接口服务标识, 请求提交方式为 POST, 参数数据格式以 JSON 封装;
- 请求参数内容:
 - 所有接口请求内容的字段名一致, 各接口业务字段放在 data 中;
- 请求方参数主要包括应用帐号 appid、data、noise、sign、version;
- 参数 data 内容:
 - data 为请求业务参数, 根据不同的 API 构造参数, 采用 BASE64 编码, 字符集 UTF-8;
 - ◆ 如果需要使用 V3.0 数据加密, 请将 data 转换成 BASE64 编码字符串后, 再进行加密。
- 签名 sign 按照 ASCII 码从小到大顺序 appid+data+noise+key(APPID 对应的 appkey 简称 key) +version 进行合并字符串做 MD5/SM2 加密。(签名规则详见-2.3 签名规则);

参数中文 名称	参数 标识	类型定义	必填	说明
应用帐号	appid	String	是	调用方应用帐号, 由平台提供
请求业务参数	data	String	是	各 API 请求业务参数的内容不同, 实际内容以各接口 API 为准; API 请求业务参数值, 参数 JSON 的格式, 并且做 BASE64 编码, 编码字符集 UTF-8; 数据加密方式根据版本号来做区分, 目前支持 SM4 加密;
请求随机标识	noise	String	是	每次请求生成一个唯一编号, 全局唯一 (举例: UUID、时间戳+随机数) 最大长度建议 32;
版本号	version	String	是	默认 2.0, 可选 2.0/3.0;
签名	sign	String	是	MD5/SM2 私钥签名;

请求参数示例

- JSON 中 data 参数明文内容:

```
{
  "appid": "7e7f4e61189145c1a5c2cce38a4219b3",
  "data": {
    "eBillBatchCode": "0100000100",
    "eBillNo": "000000001",
    "eRandom": "DFG456"
  },
  "noise": "ibuaiVcKdpRxkhJAXXXX",
  "version": "2.0",
  "sign": "E9324CF02F95CB072B6DBCEA33E725C3"
}
```

- JSON 中的 data 参数转换成 BASE64 后的最终请求内容(版本为 2.0):

```
{
  "appid": "7e7f4e61189145c1a5c2cce38a4219b3",
  "data": "ewogICAgICAgICJlcmIsbEJhdGNoQ29kZSI6IiwMTAwMDAwMTAwIiwKICAgICAgICAgIiZUJpbGx0byI6IiwMDAwMDAwMDAxIiwKICAgICAgICAgIiZVJhbmRvbSI6IiJERkc0NTYiCiB9",
  "noise": "ibuaiVcKdpRxkhJAXXXX",
  "version": "2.0",
  "sign": "E9324CF02F95CB072B6DBCEA33E725C3"
}
```

- JSON 中的 data 参数转换成 BASE64 后，再通过 SM4 加密的最终请求内容(版本为 3.0):

- 示例中：SM4 加密的秘钥：4fe15d9506a75e9a3ab5f64a5df47008

```
{
  "appid": "7e7f4e61189145c1a5c2cce38a4219b3",
  "data": "d0c7d58cd0e7540cb3cc7fe7ee96a67ba12b14dae37e30459d8499bad377ef18290f897fafa3489c289f1baa98caef72c3458e0b6036797826483271bbc20349d2822e659fb83ddc7ccf1a5b72dd79391a43a70c7c4b4b50886b4eeb041d11af2f8ef0b24dd4e2e05db76c2051c1644539e02029ccfb8c6c4fb9bf5f97988551db2fa28265026438acba33c24e1977e4",
  "noise": "ibuaiVcKdpRxkhJAXXXX",
}
```

```

"version":"3.0",

"sign":"3fe102f547232f3f2659e5e690261bea41570a41e10816d0608460c08d7bf936"

}

```

2.1.2. 服务提供者返回(响应)请求结果

● 响应参数内容：

参数中文名称	参数名	类型定义	是否必填	说明
返回结果参数	data	String	是	各 API 调用返回的内容不同，实际内容以各接口 API 为准，参数值为 JSON 格式做 BASE64 编码，编码字符集 UTF-8 ； data 中的内容： {"result":"S0000","message":"成功"}
请求随机标识	noise	String	是	每次请求返回一个唯一编号，全局唯一。请求处理方以此判断是否重复提交，如果重试或重复提交请求需要使用相同的 noise
签名	sign	String	是	MD5/SM2 私钥签名

- (1) 接口返回内容主要包括结果 data、随机标识 noise、签名 sign。其中 data 为返回结果参数，根据不同的 API 构造参数；其余的参数为公用参数；
- (2) 签名 sign 按照 ASCII 码从小到大顺序 appid+data+noise+key（秘钥）+version，进行合并字符串做摘要。（签名规则详见-2.3 签名规则）；
- (3) 服务请求方在接收到响应参数后可选进行验签、解析处理；
- (4) 在验签成功后，将 data 做 BASE64 解码，读取解码结果。
- (5) 返回参数 data 两种应答：

（一）通用失败应答

所有接口返回失败信息格式，节点：返回参数 data 内容。

参数	描述	类型	长度	必填	补充
result	返回结果标识	String	10	是	除 S0000 代码值之外的代码，都表示失败：如 E0001 等

message	返回结果内容	String	不限	是	错误信息
---------	--------	--------	----	---	------

(二) 通用成功应答

通用返回成功信息格式，节点：返回参数 data 内容。

参数	描述	类型	长度	必填	补充
result	返回结果标识	String	10	是	S0000
message	返回结果内容	String	不限	是	成功信息

2.1.2.1. 返回(响应)参数示例

1) 请求成功时，返回 json（返回结果参数 message）格式如：

a) 请求成功返回结果，请求方读取 message 参数中的内容。

```
{"result":"S0000","message":"响应业务参数"}
```

2) 请求失败时，返回 json（返回结果参数 message）格式如：

a) 请求失败返回结果，请求方读取 message 参数中的内容。

```
{"result":"E0001","message":"错误信息"}
```

3) 响应 JSON 示例：

a) 请求方只需要读取响应的 data 参数中的内容进行 BASE64 解码，读取解码结果即可：

● 响应 JSON 中 data 参数明文内容：

```
{
  "data":{"result":"E0001","message":"错误信息"},
  "noise":"ibuaiVcKdpRxkhJAX000",
  "sign":"E9324CF02F95CB072B6DBCEA33E725C3"
}
```

● 响应 JSON 中 data 参数，BASE64 转码后内容：


```
{
  "data": "eyJyZXN1bHQiOiJFMdAwMSlml1c3NhZ2UiOiLpLJnor6/kv6Hnga8ifQ==",
  "noise": "ibuaiVcKdpRxkhJA000",
  "sign": "E9324CF02F95CB072B6DBCEA33E725C3"
}
```

- 响应 JSON 中 data 参数，BASE64 转码后内容，只做 BASE64 转码，响应参数不做任何加密；
 - 由于响应参数大部分为票据详情(报文会很大)，为了节省失效，不再将响应的票据做加密！
- 所有响应参数均为 MD5 签名，响应签名规则不再根据版本号区分！

2.3.签名规则

为了防止用户信息被抓包获取后，进行信息篡改，需要对所有参数（不包括签名字段）进行签名加密。

接口调用方和接口接收方对所有的参数按参数名 ASCII 码从小到大排序，使键值对的格式 key=value 拼接字符串（键值对使用&符号拼接），对拼接字符串做 MD5/SM3 加密，生成签名串 sign。

2.3.1.签名与加密算法

- 将请求/响应参数拼接后，再通过 MD5 摘要/SM2 加密算法，得到加密的签名字符串；

特别注意以下重要规则：

- ◆ 参数名 ASCII 码从小到大排序（字典序）；
- ◆ 参数名区分大小写；
- ◆ 传送的 sign 参数不参与签名。

例如请求参数的参数如下：

```
// data 未被 BASE64 编码前
{
  "appid": "ABCDEF61001",
  "data": {
    "result": "E0001",
    "message": "错误信息"
  },
  // 或者 "data": {
  //   "result": "E0000",
  //   "message": {
  //     "invoiceCode": "1101"
  //   }
  // },
  "noise": "ibuaiVcKdpRxkhJA",
}
```

```
"version": "2.0"
}
```

```
// data 被 BASE64 编码后

{
    "appid": "ABCDEF1001",
    "data": "eyJyZXN1bHQiOiJFMdAwMSIsIm1lc3NhZ2UiOiLpLJnor6/kv6Hmga8ifQ==",
    "noise": "ibuaiVcKdpRxkhJA",
    "version": "2.0"
}
```

第一步：对参数按照 key=value 的格式，并按照参数名 ASCII 字典序排序如下：

```
stringA="appid=app1&data=eyJyZXN1bHQiOiJFMdAwMSIsIm1lc3NhZ2UiOiLpLJnor6/kv6Hmga8ifQ==&noise=ibuaiVcKdpRxkhJA";
```

第二步：拼接 API 密钥 key:

- 拼接 API
 - 将 data 参数由 JSON 字符串转换为 base64 后的拼接；

```
String SignTemp=
"appid=ABCDEF1001&data=eyJyZXN1bHQiOiJFMdAwMSIsIm1lc3NhZ2UiOiLpLJnor6/kv6Hmga8ifQ==&noise=ibuaiVcKdpRxkhJA&key=192006250b4c09247ec02f6a2d&version=2.0";
```

- 注：key 为接口提供方发放的秘钥 key，每个 APPID 对应一个 key。

第三步：MD5 加密后最终得到发送的数据【V2.0】：

- v2.0 时使用，细节参考 2.4.1

```
sign=MD5(SignTemp);
```

```
Sign="0BD492BAE4F8936E0EABAE232E1676B0";
```

```
{
    "appid": "ABCDEF1001",
    "data": "eyJyZXN1bHQiOiJFMdAwMSIsIm1lc3NhZ2UiOiLpLJnor6/kv6Hmga8ifQ==",
    "noise": "ibuaiVcKdpRxkhJA",
    "version": "2.0"
}
```

```

"version":"2.0",
"sign":"0BD492BAE4F8936E0EABAE232E1676B0"
}

```

第四步：SM2 签名后最终得到发送的数据【V3.0】

- 注：V3.0 会使用 SM2 加密签名，V2.0 及之前使用 MD5 加密签名即可；
- 细节参考 2.4.2
- privateKey 私钥由调用方提供，publicKey 公钥由调用方保存，接收方不需要知道。
- 拼接 API
 - 将 data 参数由 JSON 字符串转换为 base64，再将 BASE64 进行 SM4 加密后的拼接；

```

String SignTemp=
"appid=ABCDEFG1001&data=a3e709096c9b5eacffe24137d186ae87fb3591cfd73f43af5fa441545fcf
eacfe70e00b03e6d033e3dbd80d86ae0ebc8bf50a06edec54c453b7af3fe70413046&noise=ibuaiVcKd
pRxkhJA&key=192006250b4c09247ec02f6a2d&version=3.0";

```

```
sign=SM2(SignTemp,privateKey);
```

```
sign="MEUCIQDU2QWZikGcYt+VB4sgQbGcTJEenP7nJ9PfdQ3giAmogQlgPAI8wkPYFD0shSCHC4HCG6LT7B
8Cfy/6ia18meB+sTk=";
```

```

{
  "appid":"7e7f4e61189145c1a5c2cce38a4219b3",
  "data":"a3e709096c9b5eacffe24137d186ae87fb3591cfd73f43af5fa441545fcfeacfe70e00b03e6d033e3dbd80d86ae0ebc8bf50a06edec54c
453b7af3fe70413046&noise=ibuaiVcKdpRxkhJA",
  "noise":"ibuaiVcKdpRxkhJAXXXX",
  "version":"3.0",
  "sign":"MEUCIQDU2QWZikGcYt+VB4sgQbGcTJEenP7nJ9PfdQ3giAmogQlgPAI8wkPYFD0shSCHC4HCG6LT7B8Cfy/6ia18meB+sTk="
}

```

2.3.2.请求随机标识算法

- API 接口协议中包含字段 noise，随机不重复即可，主要保证签名不可预测，建议最大长度 32。

2.4.数据签名与加密示例

2.4.1.BASE64–MD5 数据编码与签名示例 V2.0

2.4.1.1.请求服务示例（JAVA）

```
package com.bs.test;

import com.fasterxml.jackson.databind.ObjectMapper;
import org.apache.commons.codec.binary.Base64;
import org.apache.commons.codec.digest.DigestUtils;
import org.apache.commons.lang3.StringUtils;
import org.apache.http.HttpResponse;
import org.apache.http.client.methods.HttpPost;
import org.apache.http.entity.StringEntity;
import org.apache.http.impl.client.CloseableHttpClient;
import org.apache.http.impl.client.HttpClients;
import org.apache.http.util.EntityUtils;

import java.nio.charset.Charset;
import java.util.HashMap;
import java.util.Map;

public class SignTest1 {

    public static void main(String[] args) {
        //接口地址
        String url = "https://www.chinaebill.cn/invoice-rest/xxxx";
        //应用账号
        String appId = "ABCDEFGH1001";
        //密钥 签名私钥 key
        String appKey = "192006250b4c09247ec02f6a2d";
        //接口对应的原始参数
        String data = "{\"result\":\"E0001\",\"message\":\"错误信息\"}";
        //版本号
        String version = "2.0";
        //唯一值 请求随机标识
        String noise = "ibuaiVcKdpRxkhJA";
        // 1.BASE64 加密 data 串 data 接口参数加密
        data = Base64.encodeBase64String(data.getBytes(Charset.forName("UTF-8")));
        // 2.拼接请求参数
        StringBuilder str = new StringBuilder();
```

```

        str.append("appid=").append(appID);
        str.append("&data=").append(data);
        str.append("&noise=").append(noise);
        str.append("&key=").append(appKey);
        str.append("&version=").append(version);
        // 3.MD5 加密 生成 sign
        String sign =
DigestUtils.md5Hex(str.toString().getBytes(Charset.forName("UTF-8"))).toUpperCase();
        // 4、最终请求串
        Map<String, String> map = new HashMap<>();
        map.put("appid", appID);
        map.put("data", data);
        map.put("noise", noise);
        map.put("sign", sign);
        map.put("version", version);
        try {
            HttpPost httpPost = new HttpPost(url);
            CloseableHttpClient client = HttpClients.createDefault();
            String respContent = null;
            //5.请求参数转 JSON 字符串
            StringEntity entity =
new StringEntity(new ObjectMapper().writeValueAsString(map), "utf-8");
            entity.setContentEncoding("UTF-8");
            entity.setContentType("application/json");
            httpPost.setEntity(entity);
            HttpResponse resp = client.execute(httpPost);
            if (resp.getStatusLine().getStatusCode() == 200) {
                String rev = EntityUtils.toString(resp.getEntity());
                System.out.println("返回串--》" + rev);
                Map result = new ObjectMapper().readValue(rev, Map.class);
                String rdata = result.get("data").toString();
                String rnoise = result.get("noise").toString();
                //1、拼接返回验签参数
                StringBuilder str1 = new StringBuilder();
                str1.append("appid=").append(appID);
                str1.append("&data=").append(rdata);
                str1.append("&noise=").append(rnoise);
                str1.append("&key=").append(appKey);
                str1.append("&version=").append(version);
                // 2.MD5 加密 生成 sign
                String rmd5 =
DigestUtils.md5Hex(str1.toString().getBytes(Charset.forName("UTF-8"))).toUpperCase();
                String rsign = result.get("sign").toString();
                System.out.println("验签--》" + (StringUtils.equals(rsign, rmd5)));
            }
        }
    }
}

```

```

        String busData =
new String(Base64.decodeBase64(result.get("data").toString()), "UTF-8");
        System.out.println("返回业务数据--》" + busData);
        Map busDataMap = new ObjectMapper().readValue(busData, Map.class);
        System.out.println("业务信息解密--》" +
new String(Base64.decodeBase64(busDataMap.get("message").toString()), "UTF-8"));
    } else {
        System.out.println("请求失败");
    }
} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

2.4.2.BASE64-SM4-SM2 数据加密与签名示例 V3.0

- 所有请求参数先把 JAVA 对象转换为 JSON 字符串，将 JSON 字符串通过 BASE64 编码，最终使用 BASE64 编码的 data 结果进行 SM4 加密和 SM2 签名；
 - dataSM4 加密结果 = Java 对象->JSON->BASE64 编码->SM4 加密；
 - data 明文结果 = SM4 解密->BASE64 解码->JSON->Java 对象；
- 在原有 V2.0 数据基础上进行了 data 数据加密；
 - 加密方式是在原有 data 字段转换为 BASE64 编码后，再进行加密，解密后的数据还是 BASE64 的编码数据；
 - 签名方式与 2.0 基本一样，只是把签名算法由原来的 MD5 改为 SM2，签名时用到的 data 数据由原来的 BASE64 编码结果改为 SM4 加密结果；
- 加密：SM4/ECB/PKCS5Padding
- 签 名：Signature.getInstance(GMObjectIdentifiers.sm2sign_with_sm3.toString(), BouncyCastleProvider.PROVIDER_NAME)

2.4.2.1.请求服务示例（JAVA）

- 平台提供方可以提供 java 语言的加解密工具类，和签名工具类

2.5.结果标识列表

EXXX：除了 S0000 之外的代码,其它所有值都表示错误的返回代码。

编码	描述
----	----

S0000	成功的返回代码
EXXX	除了 S0000 之外的代码,其它所有值都表示错误的返回代码。
E0025	业务异常
E0024	安全码错误(签名失败)
E0023	参数转换异常
E0022	参数错误, 请核对后重试
E0021	appid 不存在, 或者没有权限
E0019	appid 不存在或者为空
E5000	系统异常

3. 接口列表

业务分类	接口编码	接口名称	提供方	说明
个人授权	100101001	个人授权备案	平台	该接口用于保司向平台同步个人相关业务的查询授权
其他	700101001	个人健康评测	平台	根据个人授权及身份信息,依据电子票据相关费用明细,推断是否患有相关疾病,并输出命中的疾病分类名称

4. 接口详情

4.1. 个人授权

4.1.1. 个人授权备案

接口名称	个人授权备案	接口编号	100101001
------	--------	------	-----------

接口描述		该接口用于保司向平台同步个人相关业务的查询授权。			
请求业务参数（data）					
参数	中文名	类型	长度	必填	备注
name	授权人姓名	String	100	是	
idCard	授权人身份证号	String	1000	是	
phone	授权人手机号	String	11	否	
idCardType	身份证号加密方式	String	2	否	详见附录 6；默认不加密
busNo	业务流水号	String	50	是	用于标识具体的一笔业务，单位内唯一
claimCompanyCode	保险公司代码	String	30	是	统一社会信用代码
claimCompanyName	保险公司名称	String	200	是	
busType	业务类型	String	2	是	1:理赔 2:核保 3:其他
authFile	授权文件	String	不限	否	授权文件和地址必须选择一种;文件转成 byte 数组后编码为 base64 字符串； 限制 10M
authFileUrl	授权文件下载地址	String	不限	否	授权文件和地址必须选择一种
authFileType	授权文件类型	String	10	否	png/pdf/docx 等
响应业务参数（data）					
参数	中文名	类型	长度	必填	备注
result	处理结果	String	10	是	详见 2.5 结果标识

					列表
message	结果信息	节点	不限	是	节点
message 节点 (data->message)					
参数	中文名	类型	长度	必填	备注
authCode	授权码	String	30	是	
请求参数示例					
<pre>{ "name": "张三", "idCard": "123456789012345678", "phone": "13800138000", "idCardType": "0", // 身份证号加密方式，默认不加密 "busNo": "202403070001", "claimCompanyCode": "1234567890123456", "claimCompanyName": "某某保险公司", "busType": "1", // 业务类型：1-理赔 "authFile": "base64EncodedAuthFileContent", // 授权文件，转成 byte 数组后编码为 base64 "authFileUrl": "http://example.com/authfile.png", // 授权文件下载地址 "authFileType": "png" // 授权文件类型 }</pre>					
响应参数示例					
<pre>{ "result": "S0000", "message": { "authCode": "AF10001" } }</pre>					

4.2.个人健康评测

接口名称	个人健康评测	接口编号	700101001		
接口描述	根据个人授权及身份信息，依据电子票据相关费用明细，推断是否患有相关疾病，并输出命中的疾病分类名称				
请求业务参数（data）					
参数	中文名	类型	长度	必填	备注
authCode	授权码	String	30	是	
idCard	授权人身份证号	String	1000	是	
issueDateStart	评测起始就诊日期	String	10	否	yyyy-MM-dd 默认查询 2 年
issueDateEnd	评测截止就诊日期	String	10	否	
assessTypes	评测内容	String	1	是	1-风险疾病分类
响应结果业务参数（data）					
参数	中文名	类型	长度	必填	备注
result	处理结果	String	10	是	详见 2.5 结果标识列表
message	结果信息	String	不限	是	
message 节点（data->message）					
参数	中文名	类型	长度	必填	备注
hitCount	疾病分类数量	Number	6	是	
diseaseCategory	疾病分类名称	List<String>	200	否	若为空则无风险
请求参数示例					
{ "authCode": "123456", // 授权码 "idCard": "123456789012345678", // 授权人身份证号 "issueDateStart": "2023-01-01", // 评测起始就诊日期，可选参数，默认查询 2 年 "issueDateEnd": "2025-01-01", // 评测截止就诊日期，可选参数 "assessTypes": "1" // 评测内容，1-风险疾病分类 }					
响应参数示例					
{ "result": "S0000", // 处理结果 }					

```
"message": {  
  "hitCount": 3, // 疾病分类数量  
  "diseaseCategory": [ // 疾病分类名称  
    "常规慢性病",  
    "严重遗传性疾病",  
    "严重遗传性疾病"  
  ]  
}  
}
```